
----- Docker -----

◆ 1. Images & Containers

🔍 Summary:

- A **Docker image** is a **read-only template** that contains the application code, dependencies, and runtime environment.
 - A **container** is a **running instance of an image** — isolated, lightweight, and portable.
-

✅ Example:

```
docker build -t myapp:latest .  
docker run -d --name my_container myapp:latest
```

💬 Interview Q&A:

Q1: What is the difference between an image and a container? A1:

- Image = blueprint (static)
 - Container = running instance (dynamic)
-

Q2: Can you run multiple containers from one image? A2: ✅ Yes, containers are created independently from the same image.

Q3: How are containers isolated? A3: Using Linux features like **namespaces** (PID, network) and **cgroups** (resource limits).

Q4: What happens when a container is stopped? A4: It retains its filesystem but stops running; unless you remove it (`docker rm`), the state persists.

◆ 2. Docker Compose

🔍 Summary:

Docker Compose lets you define and run multi-container apps using a YAML file (`docker-compose.yml`). It simplifies orchestration and configuration.

✅ Example (`docker-compose.yml`):

```
version: '3'
services:
  web:
    build: .
    ports:
      - "8080:80"
  db:
    image: postgres
    environment:
      POSTGRES_PASSWORD: secret
```

Run with:

```
docker-compose up -d
```

💬 Interview Q&A:

Q1: What is Docker Compose used for? A1: To configure and run multiple containers (e.g., app + DB + Redis) with a single command.

Q2: How does Compose handle networking? A2: It creates a default **bridge network**, and all services can reach each other using **service names** as hostnames.

Q3: How do you override configurations for different environments? A3: Use **override files** (`docker-compose.override.yml`) or **environment variables**.

Q4: What happens when you run `docker-compose down` ? A4: It stops containers and removes networks, volumes (if `--volumes` used), and Compose state.

◆ 3. Dockerfile

Summary:

A **Dockerfile** is a script containing instructions to build an image, such as base image, installed packages, app code, and commands to run.

Example:

```
FROM node:18
WORKDIR /app
COPY . .
RUN npm install
CMD ["node", "server.js"]
```

Interview Q&A:

Q1: What are key Dockerfile instructions? A1:

- `FROM` : base image
 - `COPY` / `ADD` : copy files
 - `RUN` : execute command at build time
 - `CMD` / `ENTRYPOINT` : default runtime command
 - `EXPOSE` : declare port
 - `ENV` : set environment variables
-

Q2: What's the difference between `CMD` and `ENTRYPOINT` ? A2:

- `CMD` : sets default args
 - `ENTRYPOINT` : sets fixed command They can be combined — `ENTRYPOINT` runs with `CMD` as default args.
-

Q3: What is the build context? A3: The directory sent to the Docker daemon — it includes everything accessible to `COPY` or `ADD` .

Q4: How do you reduce image size in a Dockerfile? A4:

- Use smaller base images (e.g., `alpine`)
 - Combine commands with `&&`
 - Clean up temp files (`rm -rf /var/cache/...`)
 - Use multi-stage builds
-

◆ 4. Volumes

Summary:

Volumes are persistent storage for containers. They survive container restarts and decouple container logic from data.

Example:

```
docker volume create mydata
docker run -v mydata:/app/data myapp
```

Or in Compose:

```
volumes:
  - dbdata:/var/lib/postgresql/data
```

Interview Q&A:

Q1: Why use volumes? A1:

- Persist data beyond container lifecycle
 - Share data between containers
 - Avoid writing to container's internal FS
-

Q2: What are the volume types in Docker? A2:

- **Named volumes** (`docker volume create`)
 - **Anonymous volumes** (no name, autogenerated)
 - **Bind mounts** (`-v /host:/container`) — direct host access
-

Q3: Where are Docker volumes stored by default? A3: In `/var/lib/docker/volumes/` on the host.

Q4: Can you back up Docker volumes? A4:  Yes — mount the volume and tar it:

```
docker run --rm -v mydata:/data -v $(pwd):/backup busybox tar czf /backup/data.tar.gz /data
```

◆ 5. Networking

Summary:

Docker provides **built-in networking drivers** to manage communication between containers and external systems.

Default networks:

- **bridge** (default for standalone containers)
 - **host** (shares host network stack)
 - **none** (no network access) Custom networks allow **DNS-based service discovery**.
-

✓ Example:

```
docker network create my-net
docker run -d --name db --network my-net postgres
docker run -d --name app --network my-net myapp
```

💬 Interview Q&A:

Q1: What is the difference between bridge and host network? A1:

- **bridge**: Isolated virtual network with NAT
 - **host**: Uses host's network directly — no isolation
-

Q2: How do containers communicate in the same network? A2: By using container names as hostnames (thanks to internal DNS).

Q3: Can containers on different networks talk? A3: Not unless **you connect** them to a shared network or use **published ports**.

Q4: What happens if no `--network` is specified? A4: Container uses the **default bridge** network (with IP-based communication).

◆ 6. Health Checks

🔍 Summary:

A **HEALTHCHECK** tells Docker how to determine if a container is "healthy". Useful for orchestration, auto-restart, or load balancing.

✓ Dockerfile Example:

```
HEALTHCHECK --interval=30s --timeout=5s --retries=3 \
CMD curl -f http://localhost:3000/health || exit 1
```

Interview Q&A:

Q1: What states can a container health check return? A1:

- `starting` : before first check
- `healthy` : last check passed
- `unhealthy` : last check failed

Q2: Can health checks restart a container automatically? A2: Not by themselves — but orchestrators like Docker Compose, Swarm, or Kubernetes can act on the health state.

Q3: How do you see health check status? A3:

```
docker inspect --format='{{json .State.Health}}' <container>
```

Q4: What are good practices for health check commands? A4:

- Keep them fast and lightweight
- Use curl/netcat to ping a known port or URL
- Avoid side effects

7. Layered Architecture

Summary:

Docker images are **layered** — each Dockerfile instruction creates a **new layer**, which can be **cached and reused**, making builds efficient and modular.

✓ Visual Structure:

```
FROM python:3.10          ← base layer
COPY . /app               ← adds code
RUN pip install -r req.txt ← dependencies
CMD ["python", "main.py"] ← final layer
```

💬 Interview Q&A:

Q1: What are image layers in Docker? A1: Read-only filesystem layers created from each Dockerfile instruction.

Q2: How do layers help with build speed? A2: Docker **caches** unchanged layers, avoiding rebuilds — dramatically speeding up builds.

Q3: What happens if you change one line in a Dockerfile? A3: Docker **invalidates** the cache **from that line onward** — later layers are rebuilt.

Q4: How to optimize Dockerfile for layer reuse? A4:

- Put frequently-changing commands near the **bottom**
- Combine `RUN` steps
- Avoid unnecessary `COPY` commands

◆ 8. Entrypoint

🔍 Summary:

ENTRYPOINT defines the **main command** that always runs when a container starts. Can be used alone or with **CMD** for default arguments.

✓ Dockerfile Example:


```
ENTRYPOINT ["python", "app.py"]  
CMD ["--port=8080"]
```

Interview Q&A:

Q1: How is ENTRYPOINT different from CMD? A1:

- `ENTRYPOINT` : defines the actual **executable**
- `CMD` : provides **default arguments** If both are defined, Docker runs: `ENTRYPOINT + CMD`

Q2: How do you override CMD at runtime? A2:

```
docker run myapp --port=9090
```

But you cannot override `ENTRYPOINT` unless using `--entrypoint` .

Q3: Can ENTRYPOINT be a shell script? A3:  Yes — just make sure the script is executable and uses `#!/bin/sh` or similar.

Q4: What is the best practice for ENTRYPOINT? A4: Use `ENTRYPOINT` for the **core app binary**, and `CMD` for flexible arguments (so you can override them easily).

9. Debugging

Summary:

Debugging Docker containers involves checking logs, inspecting container state, and accessing the container's shell.

Commands:

```
docker logs <container>
docker exec -it <container> /bin/sh
docker inspect <container>
```

Interview Q&A:

Q1: How do you debug a container that crashes immediately? A1:

- Use `docker logs <container>` to view stderr/stdout
- Use `docker run -it --entrypoint sh` to override the command and inspect manually

Q2: How do you get environment variables inside a running container? A2:

```
docker exec <container> printenv
```

Q3: What does `docker inspect` show? A3: Full container metadata (IP, mounts, health, config, etc.) in JSON format.

Q4: How do you troubleshoot network issues? A4:

- Use `ping` or `curl` inside container
- Inspect network with `docker network inspect`
- Check ports, DNS, bridge network routing

◆ 10. Tagging

Summary:

Tags identify versions of a Docker image. By default, an image with no tag is considered `latest`.

Example:

```
docker build -t myapp:1.0 .
docker tag myapp:1.0 myapp:stable
```

Interview Q&A:

Q1: What is the purpose of image tags? A1: Tags allow versioning and differentiation between builds (e.g., `latest`, `dev`, `prod`, `v1.2.3`).

Q2: Can you have multiple tags for a single image? A2:  Yes, via `docker tag` you can alias an image with multiple names.

Q3: How do tags relate to container registries? A3: When pushing to a registry (e.g., Docker Hub), the tag determines how the image is labeled:

```
docker push myapp:1.0
```

Q4: Is `latest` always the latest version? A4:  No — `latest` is just a **tag name**, not automatically updated. It can be misleading if not maintained properly.

◆ 11. Compose File

Summary:

The **Compose file** (`docker-compose.yml`) defines services, networks, volumes, and configurations for Docker Compose.

Example:

```
version: '3.8'
services:
  app:
```

```
build: .
ports:
  - "3000:3000"
depends_on:
  - db
db:
  image: postgres:14
  volumes:
    - db_data:/var/lib/postgresql/data

volumes:
  db_data:
```

Interview Q&A:

Q1: What does `depends_on` do? A1: It sets **startup order**, but **not health checks** — you should use explicit health checks if required.

Q2: Can you define networks and volumes in Compose? A2:  Yes — under the `networks` and `volumes` top-level keys.

Q3: What is the role of `build` vs `image` ? A3:

- `build` : Build from a local Dockerfile
- `image` : Pull prebuilt image from registry

Q4: How do you scale services using Compose? A4:

```
docker-compose up --scale worker=5
```

◆ 12. Init System

Summary:

Containers often lack a proper **init process** (`PID 1`), which can lead to **zombie processes** and **signal handling issues**.

A container **init system** ensures clean child process management.

✓ Solutions:

- Use minimal init like `tini`

```
ENTRYPOINT ["/tini", "--", "node", "server.js"]
```

- Or use `--init` flag:

```
docker run --init myapp
```

💬 Interview Q&A:

Q1: What is the PID 1 problem in containers? A1: If the container's main process is not an init system, zombie child processes may accumulate due to improper signal handling.

Q2: What does Docker's `--init` flag do? A2: It injects a tiny init system (like `tini`) into the container to handle reaping zombie processes and forwarding signals.

Q3: How does `tini` help? A3: Acts as PID 1 inside the container, reaping zombies and handling shutdown signals like `SIGTERM`.

Q4: Is using `--init` a best practice? A4: ✓ Yes, especially for containers that spawn child processes or run daemons.

◆ 13. Overlay Networks

Summary:

Overlay networks allow containers running on **different Docker hosts** (in a Swarm or Kubernetes cluster) to communicate securely as if they were on the same local network.

Example (Docker Swarm):

```
docker network create -d overlay my_overlay
```

Use in `docker-compose.yml` or service definitions.

Interview Q&A:

Q1: What is an overlay network? A1: A software-defined network that connects containers across multiple hosts using tunneling and encryption.

Q2: When do you need an overlay network? A2: In **distributed environments** — like Docker Swarm, Kubernetes, or multi-host Compose setups.

Q3: How is traffic routed in overlay networks? A3: Using **VXLAN encapsulation**, Docker tunnels traffic between hosts participating in the network.

Q4: Can containers on overlay networks access the internet? A4:  Yes, if NAT or a gateway is configured (e.g., via Swarm ingress routing or host-level NAT).

14. Security Best Practices

Summary:

Securing Docker involves **minimizing attack surface**, **limiting privileges**, and **auditing containers and images**.

✓ Common Practices:

- Use **official** base images
 - Run as **non-root** inside containers
 - Use **read-only** file systems
 - Scan images for CVEs (e.g., `docker scan`, Trivy)
 - Drop unnecessary capabilities
-

💬 Interview Q&A:

Q1: Why avoid running containers as root? A1: If a container breakout occurs, root user privileges can allow full host access.

Q2: How can you enforce read-only containers? A2:

```
docker run --read-only ...
```

Q3: What tools help scan for vulnerabilities in Docker images? A3:

- `docker scan`
 - Trivy
 - Snyk
 - Clair
-

Q4: What Dockerfile practices improve security? A4:

- Use slim or alpine base images
 - Avoid secrets in `ENV` or image layers
 - Use multi-stage builds to remove build tools
-

◆ 15. Ports

Summary:

Ports expose container services to the host or other containers. Docker maps **container ports** to **host ports** using `-p` or `--publish`.

Example:

```
docker run -p 8080:80 myapp
```

This maps **host port 8080** → **container port 80**

Interview Q&A:

Q1: Difference between `-p` and `--expose` ? A1:

- `-p` (publish) = exposes port to host
 - `--expose` = documents intent; doesn't publish
-

Q2: Can multiple containers use the same host port? A2:  No, unless on different IPs or using different interfaces (e.g., `127.0.0.1:8080` and `0.0.0.0:8080`).

Q3: What happens if you run without `-p` ? A3: The service is only accessible **inside the Docker network**, not from the host.

Q4: What's the format of port mapping? A4: `[host_ip:]host_port:container_port` , e.g.:

```
-p 127.0.0.1:8080:80
```

16. Image Caching

Summary:

Docker **cached** image layers during builds. If a layer hasn't changed, it's **reused**, improving build speed.

✓ Optimization Example:

```
# Good order to enable caching
COPY package.json ./
RUN npm install
COPY . .
```

💬 Interview Q&A:

Q1: When is Docker's cache invalidated? A1: From the first changed line onward in the Dockerfile — all subsequent layers are rebuilt.

Q2: How to maximize Docker layer caching? A2:

- Order layers from least to most likely to change
 - Separate dependency install from app code
 - Avoid unnecessary `COPY . .`
-

Q3: How can caching be harmful if misused? A3: You may unintentionally use **outdated packages** or **bypass a critical change** if cache isn't invalidated.

Q4: What command skips cache during build? A4:

```
docker build --no-cache -t myapp .
```

◆ 17. Base Image

🔍 Summary:

A **base image** is the starting point of a Dockerfile (`FROM`). It defines the underlying OS, language runtime, or framework (e.g., `python:3.10` , `alpine` , `debian`).

✓ Common Examples:

```
FROM ubuntu:22.04
FROM node:18-alpine
FROM scratch # For fully minimal custom builds
```

💡 Interview Q&A:

Q1: What is a minimal base image? A1: An image with only the bare minimum — like `alpine` or `scratch` — reducing size and surface area.

Q2: Why use `scratch` ? A2: It's an **empty base**, used for Go or Rust static binaries to create ultra-small and secure containers.

Q3: When would you choose `debian` over `alpine` ? A3: `debian` includes more libraries and tooling — good for debugging or compatibility. `alpine` is smaller but might cause issues with certain binaries due to `musl` instead of `glibc` .

Q4: Best practices when choosing base image? A4:

- Use official and versioned tags
 - Keep images minimal
 - Avoid `latest` to ensure reproducibility
-

◆ 18. Container Registries

🔍 Summary:

A **container registry** is a place to **store, manage, and distribute Docker images**, such as Docker Hub, AWS ECR, GitHub Container Registry, or self-hosted Harbor.

✓ Common Commands:

```
docker login
docker tag myapp myregistry.com/myapp:1.0
docker push myregistry.com/myapp:1.0
docker pull myregistry.com/myapp:1.0
```

💬 Interview Q&A:

Q1: What is the difference between Docker Hub and ECR? A1: Docker Hub is public (or private with limits). ECR is AWS-managed, private by default, and integrated with IAM.

Q2: Can you host your own registry? A2: ✓ Yes — using [Docker Registry](#) or platforms like **Harbor**, **Nexus**.

Q3: Why is tagging important when pushing to a registry? A3: The **tag** acts as the version — `latest`, `1.0`, etc. It determines how the image is pulled or deployed.

Q4: How do you authenticate with a private registry? A4:

```
docker login myregistry.com
```

And pass credentials (token/password).

◆ 19. Container Health

🔍 Summary:

Beyond basic **healthchecks**, monitoring container health involves tracking logs, metrics, and lifecycle events — often using tools like Prometheus, Grafana, or ECS/K8s health probes.

✓ Health Monitoring Methods:

- `docker inspect`
 - Docker health checks
 - Logging (stdout/stderr)
 - Alerts via monitoring tools (e.g., Prometheus exporters, ELK stack)
-

💬 Interview Q&A:

Q1: How do you know if a container is "healthy"? A1: If it has a `HEALTHCHECK` and returns 0 on check command — visible via `docker ps` or `docker inspect`.

Q2: What happens when a container is unhealthy? A2: Docker won't restart it automatically — but orchestrators (like Docker Swarm or Kubernetes) can respond by restarting or replacing it.

Q3: What tools are used to monitor containers? A3:

- Prometheus + Grafana
 - cAdvisor
 - Datadog, New Relic, ELK stack
-

Q4: What events might signal poor container health? A4:

- High restart count
 - Constant crash loops
 - Non-200 responses from health endpoints
 - Memory/CPU throttling
-

◆ 20. Multistage Builds

Summary:

Multistage builds allow you to use multiple `FROM` statements in a Dockerfile, enabling you to **build in one stage** and **copy the final output** to a clean runtime stage. This reduces image size and attack surface.

Example:

```
# Build stage
FROM node:18 as builder
WORKDIR /app
COPY . .
RUN npm install && npm run build

# Final stage
FROM nginx:alpine
COPY --from=builder /app/dist /usr/share/nginx/html
```

Interview Q&A:

Q1: Why use multistage builds? A1: To separate build dependencies from runtime — result is a smaller, faster, more secure image.

Q2: Can you have more than 2 stages? A2:  Yes — any number. You can name them (`as build` , `as test` , etc.) and reference as needed.

Q3: Do multistage builds affect caching? A3: Yes — each stage has its own cache chain. You can optimize by ordering `COPY` and `RUN` commands properly.

Q4: What's the difference between multi-container and multistage? A4:

- **Multi-container:** Different services (e.g., db + web)
 - **Multistage:** One image built in steps
-

Excellent choice! Here's your **Bonus Round – 10 must-know advanced/interview-critical Docker topics** not covered earlier but highly valuable for senior backend/devops/data engineering interviews.

Bonus Docker Topics (10 Advanced/Overlooked Gems)

◆ 1. Docker Contexts

📌 **Summary:** Enables switching between local and remote Docker environments.

💻 **Example:**

```
docker context create remote --docker "host=ssh://user@host"  
docker context use remote
```

💬 **Q:** Why use Docker contexts? **A:** To seamlessly manage and deploy to multiple Docker environments (e.g., dev, staging, prod).

◆ 2. BuildKit

📌 **Summary:** New build engine that improves build performance, caching, and security.

🔧 **Enable:**

```
DOCKER_BUILDKIT=1 docker build .
```

💬 **Q:** What advantages does BuildKit offer? **A:** Parallel builds, secrets management during build, efficient caching.

◆ 3. Docker Swarm

📌 **Summary:** Docker's native orchestration for clustering and service scaling.

💬 **Q:** How does Swarm differ from Kubernetes? **A:** Swarm is simpler, built-in to Docker, but less feature-rich than Kubernetes.

◆ 4. Docker Secrets

📌 **Summary:** Manage sensitive data (e.g., API keys) securely in Swarm.

💬 **Q:** How do Docker secrets work? **A:** Stored encrypted, mounted into containers at runtime (e.g., `/run/secrets/...`).

◆ 5. Logging Drivers

📌 **Summary:** Control how container logs are collected and where they are sent.

💬 **Q:** Common logging drivers? **A:** `json-file`, `syslog`, `fluentd`, `awslogs`, `gelf`, etc.

◆ 6. Resource Limits

📌 **Summary:** Limit container CPU and memory usage.

🔧 **Example:**

```
docker run --memory=512m --cpus="1.5" myapp
```

💬 **Q:** Why set resource limits? **A:** Prevent containers from starving the host or affecting other containers.

◆ 7. Docker Events

📌 **Summary:** Listen to real-time events from Docker Engine.

💬 **Q:** Use case for `docker events`? **A:** Trigger CI/CD jobs, audits, auto-scaling, or alerts on container changes.

◆ 8. Lifecycle Hooks

📌 **Summary:** Manage behavior on `start`, `stop`, or `restart` via signals (`SIGTERM`, `SIGKILL`, etc.)

💬 **Q:** What should apps do on `SIGTERM`? **A:** Cleanly close connections, flush logs, and gracefully shut down.

◆ 9. Overlay File Systems (AUFS / Overlay2)

✚ **Summary:** File system used by Docker to manage image layers.

💬 **Q:** Difference between `aufs` and `overlay2` ? **A:** `overlay2` is the default (modern, more efficient), while `aufs` is older.

◆ 10. Immutable Infrastructure

✚ **Summary:** Docker promotes the **immutable infrastructure pattern** — rather than updating running systems, you **rebuild and redeploy** containers.

💬 **Q:** Why is immutability important? **A:** Prevents configuration drift, enables rollbacks, and improves reproducibility.
